

Abstract

本文提出了 WiscKey，基於 LSM Tree 所改良的 persistent key-value store，透過將 keys 與 values 分離來最小化 I/O amplification。這種設計針對 SSD 架構進行優化，並透過 YCSB workload 與 microbenchmark 對 WiscKey 進行評測。結果發現，WiscKey 在 6 種 YCSB workload 中皆優於 LevelDB 與 RocksDB，在 load database 時能比 LevelDB 快 2.5~111 倍，而 random lookup 則快 1.6~14 倍。

Introduction

首先論文介紹了 LSM-Trees(Log-Structured Merge Trees)，其會將 key-value pairs 的寫入請求進行分批暫存，整理好後再以 sequential write 的方式去對每批請求處理，而為了高效率的 lookup 操作，LSM-Trees 也會持續地對樹內資料進行 read 與 sort。這種優勢主要體現在 HDDs 上，因 HDD 對於 random I/O 的速度比 sequential I/O 還要慢超過 100 倍。在 LSM-Trees 中同一數據會被重複讀寫，導致 I/O amplification 達到 50 倍或更高。

然而 LSM-Tree 的設計對於 SSDs 來說並不適合，因為 SSDs 有以下幾點特性：

1. SSD 對於 random I/O 和 sequential I/O 的速度差距並沒有像 HDD 那麼大。
2. SSD 有 parallelism 的處理能力，原先的 LSM-Tree 沒辦法發揮這個優勢
3. SSD 的壽命問題，LSM-Tree 的高 I/O amplification 導致 SSD 更快 wear-out。

因此這篇論文的 WiscKey 就是以 SSD 為優化目標所設計的 key-value storage system，將 key 與 value 分開，key 會儲存在 LSM tree 而 value 則存在 log 中，以此減少不必要的 value 搬動，並保持 LSM Tree 的優勢。

文中提到，WiscKey 在某些極端情況不比傳統 LSM Tree 好，像是小規模 random write，且大規模範圍的 sequential search，然而這些情況並不符合現實世界的狀況，因此 WiscKey 基本上還是成功克服了 LSM Tree 在 SSD 上的困境。

Background

這段主要講述了 LSM Tree 由多個 component 組成，從 C_0 排到 C_k ， C_0 是常駐在記憶體中的，其餘的都存在硬碟中。而其運作機制很像是多層 cache 的概念，當 C_0 裡的資料寫到接近大小限制後，就會將 C_0 與 C_1 的資料進行合併，稱為 compaction，而合併後會取代舊的 C_1 ，這個概念很像是 primary cache 資料會不時把不重要的資料 evict 到速度較慢但容量較大的 secondary cache。

而搜尋時就是從 C_0 一路搜到 C_i ，因此資料若在很下層，latency 就會比較高。後面也介紹了 LevelDB 與 SSTable 結構，不過概念都差不多，唯一比較需要注意的是壓縮(compaction)可以 background 執行，從而避免執行過程中發生

compaction 導致停滯。

LevelDB 在大量資料中的讀寫放大皆很嚴重，在 100GB 左右時寫入放大為 14，而讀取放大為 327，不過由於其能維持 sequential I/O 的存取，對於硬碟來說還是合理的 trade off，但對於 SSD 來說就得重新設計了，因 I/O amplification 實在是過於嚴重。

WiscKey

為了最小化 I/O amplification，作者提出了 WISCKEY：

1. WiscKey 將 keys 與 values 分離，僅將 keys 保留在 LSM 樹中，而將 values 儲存在單獨的 log 中。
2. 在處理 unsorted values 時，WiscKey 會利用 SSD 的 parallel random-read 特性。
3. WiscKey 會採用 unique crash consistency 與 Garbage Collection 來高效管理 value log。
4. 為了最佳化效能，WiscKey 會在不犧牲 consistency 的情況下移除 LSM Tree 的 log，減少小量 writes 時的 system-call overhead。

最顯著的改動在於 LSM Tree 只保留 key 和 value log 的位置資訊，而不直接將 value 塞在 LSM Tree 中，因此整顆 Tree 的大小會遠小於 LevelDB，同時顯著降低了 write amplification，因為只有 Key 在 compaction 的過程中會被重複寫入，而 value log 則不受影響(只寫 1 次)。

而由於 value 從 LSM Tree 中分離，對於查詢效率也會加快，因為查詢過程中需要搜索的文件數量更少了，能夠讓更多內容放在記憶體中，加快搜尋速度。

WiscKey 在儲存 Value 於 vLog 時，也會同時儲存相應的 Key，藉此來輔助 Garbage collection 在回收已被刪除或覆寫的 key value 時能夠快速檢查 Value 的有效性並決定是否回收，同時使用 tail 和 head 來控制需要檢查的範圍。

對於密集性小型寫入的問題，WiscKey 做了一個優化，會先將需要寫入到 Vlog 的 value 儲存在使用者空間的 Buffer，當 Buffer 超過某個閾值大小後才搬移到 vLog 中，使其轉變成大型寫入，從而減少 additional overhead。

Evaluation

作者主要是針對以下項目進行實驗探討：

1. Key-value 的分離對於讀寫放大與效能影響
2. Parallel range query 是否適用在現代 SSD 上
3. GC 對於 WiscKey 的效能影響
4. WiscKey 的 crash consistency、復甦時間
5. CPU overhead of WiscKey

6. WiscKey 在面對現實情境 workload 的效能比較

對於 Load performance 的比較，實驗方式為固定 key 大小，並調整 value 大小。結果發現 WiscKey 的寫入放大表現良好，在 value size 超過 1KB 後降到了 1，且 Throughput 和 tail latency 均表現比 LevelDB 好。

對於 Query Performance 的比較，實驗方式為隨機載入 100GB 的 DB，並執行 10 萬次的 read 查詢操作。對於隨機查詢而言，WiscKey 在做 compaction 時對 I/O overhead 較低，同時避免了大量的 background read/write，從而產生效能優勢與延遲較低。而範圍查詢中，WiscKey 在大 key-value pair 中性能比 LevelDB 好，但小的 key-value pair 會受限於 parallel random read 的 throughput 限制，反而比 LevelDB 慢。而在循序載入 DB 的情況中，WiscKey 的表現比 LevelDB 好。

對於 Crash Consistency 的比較，實驗方式為使用 ALICE 工具來模擬系統崩潰，測試超過 3000 個崩潰情境，結果說明 WiscKey 的一致性沒有漏洞，不過恢復時間比 LevelDB 更慢，主要原因是掃描 vLog 造成的 overhead。這部分可以依靠更頻繁地去 persist head pointer 來改善。

對於 CPU overhead 的比較，實驗比較了 sequential write、random load 和 range query。在 sequential write 中，LevelDB 的 CPU 使用率較高，因為寫入 key-value pair 到 log 文件需要 encoding 而導致的。而 WiscKey 去除了 log file 來優化寫入，因此使用率較低。在 range query 中，WiscKey 會使用高達 32 個 thread 來預先讀取，導致 CPU 使用率高於 LevelDB。

對於現實情境的壓力測試，作者採用 YCSB Benchmarks 來比較 WiscKey、LevelDB 與 RocksDB。為了測量 WiscKey 最差的效能，GC 會始終在 background 中執行。對於 loading 且 value 為 1KB 的情況下，WiscKey 的性能至少是其他 DB 的 45 倍 (最差的情況) 以上。而 value 上升到 16KB 時，WiscKey 性能是其他 DB 的 104 倍。

對於 Reads，由於資料採用 Zipf distribution 而使 WiscKey 的優勢相對變少，但 RocksDB 和 LevelDB 仍然沒有達到 WiscKey 的水準。

而在 range queries 的 workload 中，WiscKey 的效能仍然比其他兩個 DB 更好，不論 value size 的大小。

My thought

這篇論文實驗做得很豐富，且在 WiscKey 已表現良好的情況下，仍然在繼續探討可改進空間(例如 GC)。WiscKey 的優勢主要體現在 SSD 的 high parallel read throughput，也許某些較低階的設備會無法發揮相同優勢。有個小問題是在討論

系統崩潰後復甦的地方，WiscKey 在 crash 後的恢復時間仍然比 LevelDB 更高，文中雖有提及能透過更頻繁去 persist head pointer 去優化，但沒有相關實驗，不過這不影響 WiscKey 方法所帶來的價值。